PLAN DE ACCIÓN DE MEJORA

Competencia: 38368 - Desarrollar la solución de software de acuerdo con el diseño y metodologías de desarrollo.

Resultado de aprendizaje: 593108-04 Codificar el software de acuerdo con el diseño establecido.

Objetivo: Lograr que el aprendiz codifique una aplicación web completa siguiendo los lineamientos de diseño, aplicando buenas prácticas de programación, implementando una base de datos relacional, desarrollando una API y documentando dicha API de manera estructurada.

Estrategias de Mejora

1. Reforzar el análisis previo a la codificación
    - Estudio de requerimientos funcionales y no funcionales.
    - Definición de HU y casos de uso completos.
    - Construcción del modelo de datos relacional.
2. Implementación práctica
    - Desarrollo del backend con API REST.
    - Construcción del frontend que consuma la API.
    - Configuración de la base de datos relacional con integridad referencial. 3. Buenas prácticas de desarrollo
    - Código modular, uso de control de versiones con Git/GitHub.
    - Validación de datos y seguridad básica (JWT, bcrypt).
4. Documentación de la API
    - El aprendiz deberá redactar la documentación de endpoints (método HTTP, URL, parámetros, body, respuestas esperadas).
    - Puede emplear herramientas como Swagger, Postman o un documento técnico estructurado

CASO DE ESTUDIO

APLICACIÓN WEB: SISTEMA DE GESTIÓN DE CITAS MÉDICAS ONLINE

1. Descripción General

El objetivo es desarrollar una aplicación web que permita a pacientes registrarse, agendar, consultar y cancelar citas médicas con diferentes especialistas.

Los médicos podrán gestionar sus horarios disponibles y los administradores

supervisar el sistema.

El sistema se desarrollará con arquitectura cliente-servidor, a continuación algunos lenguajes con los cuales podrás trabajar, de acuerdo a tu comodidad o manejo:

• Frontend (cliente web): React / Angular / Vue o HTML + CSS + JS.
• Backend (servidor y API): Node.js con Express / Python Django / Java Spring Boot. • Base de datos relacional: MySQL o PostgreSQL.

2. Historias de Usuario (HU)

Cada historia incluye: ID, descripción y criterios de aceptación.

1. HU1 – Registro de paciente

Como paciente quiero registrarme en el sistema con mis datos personales para poder acceder a la plataforma y agendar citas.

• Criterios de aceptación:
  - El sistema debe solicitar: nombre completo, documento, correo electrónico, teléfono y contraseña.
  - La contraseña debe tener mínimo 8 caracteres, 1 mayúscula, 1 número y 1 carácter especial.
  - El correo debe validarse en formato correcto y no repetirse.
  - Después de registrarse, el usuario recibe un correo de

confirmación. 2. HU2 – Login de paciente

Como paciente quiero iniciar sesión con mi correo y contraseña para acceder a mis citas y datos personales.

• Criterios de aceptación:
  - El login debe validar que el correo existe y la contraseña coincida.
  - Si el usuario se equivoca 3 veces, se bloquea la cuenta por 15 minutos.
  - Debe permitir recuperar contraseña vía correo.

3. HU3 – Agendar cita médica

Como paciente quiero seleccionar un especialista y un horario disponible para reservar una cita médica en línea.

• Criterios de aceptación:

- El sistema debe mostrar solo horarios disponibles (sin choques con otras citas). - El paciente debe poder filtrar por especialidad y médico.
- Una vez confirmada la cita, debe guardarse en la base de datos y enviarse correo de confirmación al paciente.
- No se puede agendar más de una cita a la misma hora.

4. HU4 – Consultar mis citas

Como paciente quiero ver un listado de mis citas programadas para saber mis próximas atenciones médicas.

• Criterios de aceptación:
- El listado debe mostrar fecha, hora, médico y especialidad.
- Debe permitir filtrar citas por estado: pendientes, realizadas, canceladas. - El paciente puede descargar un comprobante en PDF

5. HU5 – Cancelar cita

Como paciente quiero cancelar una cita ya agendada para liberar el horario y permitir que otro paciente lo reserve.

• Criterios de aceptación:
- El sistema debe permitir cancelar hasta 24 horas antes de la cita. - Al cancelar, debe actualizarse el estado en la base de datos y liberarse el horario.
- El paciente recibe un correo confirmando la cancelación.

6. HU6 – Gestión de horarios del médico

Como médico quiero definir mis horarios disponibles para que los pacientes puedan reservar citas en esos espacios.

• Criterios de aceptación:
- El médico debe poder agregar franjas horarias (día y hora).
- No debe permitir horarios superpuestos.
- Solo las franjas activas estarán disponibles para los pacientes

7. HU7 – Administración del sistema

Como administrador quiero gestionar usuarios y citas para mantener la correcta operación de la plataforma.

• Criterios de aceptación:
  - El administrador puede listar pacientes, médicos y citas.
  - Puede bloquear o eliminar usuarios.
  - Puede consultar estadísticas (número de citas por especialidad, cancelaciones, etc.).

3. Casos de Uso

CU-01 Registrar paciente

Actores: Paciente (primario).

Disparador: El usuario selecciona “Crear cuenta”.

Precondiciones:

- No existe un usuario previo con el mismo correo/documento.

Postcondiciones (éxito): Usuario (rol=paciente) creado y activo; perfil de paciente creado. Postcondiciones (falla): No se crea registro; se informa el motivo.

Flujo básico:

1. Paciente ingresa nombre, documento, correo, teléfono, contraseña. 2. Sistema valida formato de correo y fortaleza de contraseña (≥8, mayúscula, dígito, especial).
3. Sistema verifica unicidad de correo/documento.
4. Sistema cifra contraseña y crea usuarios(id_rol=paciente).
5. Sistema crea pacientes(id_paciente=usuarios.id_usuario).
6. Sistema confirma registro (opcional: envía correo de bienvenida). Alternos / Excepciones:

A1: Correo o documento ya existen → mostrar error y no crear.

E1: Error de BD → registrar en logs y devolver mensaje genérico.

Reglas:

R1: Contraseña debe cumplir política.
R2: Correo y documento son únicos.

Tablas: usuarios, pacientes.

CU-02 Autenticar usuario

Actores: Paciente / Médico / Admin.

Disparador: Usuario envía correo y contraseña.

Precondiciones: Usuario activo.

Postcondiciones (éxito): Se emite JWT con rol y id_usuario.

Postcondiciones (falla): Intento fallido registrado; posible

bloqueo. Flujo básico:

1. Usuario envía credenciales.
2. Sistema verifica existencia de correo y estado activo.
3. Sistema compara password_hash.
4. Sistema genera JWT (exp. 60–120 min) y lo

devuelve. Alternos / Excepciones:

A1: 3 intentos fallidos consecutivos → bloquear 15 min.

E1: Usuario inactivo → 403.

Reglas:

R1: Control de intentos (contador por IP/usuario).

R2: JWT firmado y con expiración.

Tablas: usuarios (lectura), (opcional) registro de intentos/logs.

CU-03 Recuperar contraseña (opcional)

Actores: Cualquier usuario.

Disparador: Clic en “Olvidé mi contraseña”.

Precondiciones: Correo registrado.

Postcondiciones (éxito): Se genera token de recuperación y se envía

enlace. Flujo básico:

1. Usuario ingresa correo.
2. Sistema crea tokens_recuperacion (token único, expira_en
~1h). 3. Sistema envía correo con enlace.
4. Usuario establece nueva contraseña → invalidar token

(usado=true). Excepciones: Token expirado/ya usado → 400.

Tablas: tokens_recuperacion, usuarios.

CU-04 Gestionar especialidades del médico

Actores: Médico (primario), Admin (secundario).

Disparador: Médico abre “Mis especialidades”.

Precondiciones: Usuario con rol=medico existente.

Postcondiciones (éxito): Se actualizan vínculos en

medico_especialidad. Flujo básico:

1. Sistema muestra catálogo especialidades.
2. Médico selecciona una o varias.
3. Sistema inserta/upserta en medico_especialidad.
4. Sistema confirma operación.

Alternos / Excepciones:

A1: Especialidad repetida → ignorar con idempotencia.

E1: Médico inactivo → 403.

Reglas:

R1: Solo médico/admin puede modificar su set.

R2: PK compuesta evita duplicados.

Tablas: medico_especialidad, especialidades.

CU-05 Definir disponibilidad (crear horarios)

Actores: Médico (primario).

Disparador: Médico selecciona “Agregar disponibilidad”.

Precondiciones: Médico activo.
Postcondiciones (éxito): Slots creados en horarios con

disponible=true. Flujo básico:

1. Médico elige fecha, hora_inicio, hora_fin (ej.: 09:00–12:00) y duración de

slot (ej.: 30 min).

2. Sistema divide el rango en slots [09:00–09:30], [09:30–10:00], … 3. Para cada slot, sistema valida que no exista overlap con horarios del mismo médico.
4. Sistema crea registros horarios con disponible=true.
5. Sistema confirma.

Alternos / Excepciones:

A1: Overlap detectado → no crear esos slots, informar cuáles.

E1: Hora_fin ≤ hora_inicio → 400 (violación CHECK).

Reglas:

R1: Prohibir solapes (validación por rangos en backend).

R2: Tamaño de slot configurable (15/20/30 min).

Tablas: horarios.

CU-06 Buscar médico/horarios por especialidad

Actores: Paciente.

Disparador: Paciente abre “Agendar cita”.

Precondiciones: Existen médicos con disponible=true en horarios.

Postcondiciones (éxito): Se listan horarios disponibles para

reservar. Flujo básico:

1. Paciente filtra por especialidad y/o médico.
2. Sistema consulta medico_especialidad, medicos, usuarios.
3. Sistema muestra slots disponibles (horarios.disponible=true, fecha ≥ hoy). 4. Paciente elige un slot y continúa a UC-07.

Alternos / Excepciones:

A1: Sin resultados → mostrar mensaje y sugerir otras

fechas/especialistas. Reglas:

R1: No mostrar horarios.disponible=false.

R2: No mostrar fechas pasadas.

Tablas: medico_especialidad, especialidades, horarios, medicos, usuarios

CU-07 Agendar cita

Actores: Paciente (primario).

Disparador: Clic en “Reservar” sobre un horario.

Precondiciones: Paciente autenticado; slot disponible=true.

Postcondiciones (éxito): Se crea citas (estado=pendiente o confirmada según política), horarios.disponible=false, entrada en auditoria_citas.

Flujo básico:

1. Paciente confirma el slot y (opcional) ingresa motivo.
2. Sistema verifica que horarios.disponible=true.
3. Sistema crea citas(id_paciente, id_horario, id_estado=pendiente, motivo, fecha_creacion=now()).
4. Sistema marca horarios.disponible=false.
5. Sistema registra auditoría evento='creada'.
6. Sistema confirma y envía correo de confirmación.

Alternos / Excepciones:

A1: Condición de carrera (otro paciente reservó antes) → detectar UNIQUE (id_horario) violado → informar “slot no disponible”.

A2: Política de límite de citas activas por paciente (si aplica)

→ 409. E1: Error de BD/servicio correo → crear cita igual, loggear

incidente. Reglas:

R1: Máx. 1 cita por id_horario (garantizado por UNIQUE).

R2: El backend debe usar transacción: crear cita + marcar horario no

disponible. Tablas: citas, horarios, auditoria_citas.

CU-08 Cancelar cita

Actores: Paciente (primario), Médico/Admin (alterno).

Disparador: Clic en “Cancelar” en una cita futura.

Precondiciones: Cita en estado pendiente o confirmada; quedan ≥ 24 horas

para la hora_inicio del horario.

Postcondiciones (éxito): citas.estado=cancelada, horarios.disponible=true, registro en auditoría.

Flujo básico:

1. Actor solicita cancelar y confirma motivo/opcional.
2. Sistema valida ventana de cancelación (≥24h).
3. Sistema actualiza citas.id_estado=4 (cancelada), cancelada_por y fecha_cancelacion=now().
4. Sistema marca horarios.disponible=true.
5. Sistema audita evento='cancelada'.
6. Sistema envía correo.

Alternos / Excepciones:

A1: <24h restantes → 409 “Fuera de plazo” (mostrar política).

A2: Cita ya cancelada o realizada → 409.

E1: Fallo al reabrir horarios → revertir transacción.

Reglas:

R1: Ventana de cancelación configurable (ej. 24h).

R2: Admin puede forzar cancelación sin ventana (con motivo obligatorio). Tablas: citas, horarios, auditoria_citas

CU-09 Consultar mis citas (paciente)

Actores: Paciente.

Disparador: “Mis citas”.

Precondiciones: Autenticación válida.

Postcondiciones (éxito): Se lista historial y próximas citas.
Flujo básico:

1. Paciente solicita listado.
2. Sistema consulta citas uniendo horarios (fecha/hora) y usuarios/medicos. 3. Sistema permite filtros por estado y rango de fechas.
4. Sistema retorna resultados ordenados por fecha.

Alternos / Excepciones:

Sin resultados → lista vacía.

Reglas:

R1: Solo ver citas del propio paciente.

Tablas: citas, horarios, medicos, usuarios

CU-10 Registrar atención (marcar realizada) / No asistencia

Actores: Médico (primario), Admin (alterno).

Disparador: Finaliza la consulta.

Precondiciones: Existe cita con hora_inicio pasada y estado pendiente/confirmada. Postcondiciones (éxito): citas.id_estado pasa a realizada o no_asistio; se audita. Flujo básico (realizada):

1. Médico abre detalle de la cita.
2. Selecciona “Marcar realizada” y guarda notas (opcional).
3. Sistema actualiza id_estado=3 (realizada) y fecha_actualizacion=now(). 4. Sistema audita evento='realizada'.

Flujo alterno (no asistencia):

1. Médico verifica ausencia del paciente.
2. Selecciona “No asistió”.
3. Sistema actualiza id_estado=5 (no_asistio) y audita.

Reglas:

R1: Solo médico del horario o admin pueden cambiar a realizada/no_asistió. R2: No se puede cambiar estado de citas canceladas.

Tablas: citas, auditoria_citas.

CU-11 Administrar usuarios (activar/bloquear)

Actores: Admin.

Disparador: Panel de administración.

Precondiciones: Admin autenticado.

Postcondiciones: Cambios persistidos en

usuarios.activo. Flujo básico:

1. Admin lista usuarios con filtros (rol, estado).
2. Selecciona usuario y acción (activar/bloquear).
3. Sistema aplica cambio y confirma.

Reglas:

R1: Bloquear impide autenticación (UC-02).

Tablas: usuarios.

CU-12 Reportes operativos

Actores: Admin.

Disparador: “Reportes”.

Precondiciones: Datos existentes.

Postcondiciones: Se muestra reporte y (opcional)

descarga CSV. Flujo básico:

1. Admin elige tipo de reporte:
2. Citas por especialidad y estado en rango de fechas. 3. Tasa de no asistencia por médico.
4. Sistema agrega datos (citas + horarios + medico_especialidad). 5. Muestra tabla con totales y permite exportar.

Reglas:

R1: Reportes solo lectura, no afectan estado.

Tablas: lectura de citas, horarios, medico_especialidad, especialidades

4. Modelo de Datos Relacional (MySQL / PostgreSQL)

Roles

- id_rol PK (smallint)
- nombre (varchar(30), único) — admin, medico, paciente

Usuarios

- id_usuario PK (bigserial)
- correo (varchar(150), único, no nulo)
- password_hash (varchar(255), no nulo)
- nombre_completo (varchar(150), no nulo)
- documento (varchar(30), único)
- telefono (varchar(30))
- id_rol FK → roles.id_rol (no nulo)
- activo (boolean default true)
- created_at (timestamp default now())

pacientes

• id_paciente PK, FK (bigint) → usuarios.id_usuario (ON DELETE CASCADE) • fecha_nacimiento (date)
• sexo (char(1))
• eps (varchar(120))
• alergias (text)

médicos

• id_medico PK, FK (bigint) → usuarios.id_usuario (ON DELETE CASCADE) • registro_profesional (varchar(60))
• consultorio (varchar(60))
• estado (varchar(20) default 'activo')

especialidades

• id_especialidad PK (smallserial)
• nombre (varchar(120), único)

medico_especialidad

• id_medico FK → medicos.id_medico
• id_especialidad FK → especialidades.id_especialidad

PRIMARY KEY (id_medico, id_especialidad)

Horarios

• id_horario PK (bigserial)
• id_medico FK → medicos.id_medico (no nulo)
• fecha (date, no nulo)
• hora_inicio (time, no nulo)

• hora_fin (time, no nulo)
• disponible (boolean default true)
• created_at (timestamp default now())

Restricciones:

• CHECK (hora_fin > hora_inicio)
• UNIQUE (id_medico, fecha, hora_inicio, hora_fin) (evita traslapes

exactos) **estados_cita**

• id_estado PK (smallint) — 1=pendiente, 2=confirmada, 3=realizada,
4=cancelada, 5=no_asistio
• nombre (varchar(40), único)

citas

• id_cita PK (bigserial)
• id_paciente FK → pacientes.id_paciente (no nulo)
• id_horario FK → horarios.id_horario (no nulo)
• id_estado FK → estados_cita.id_estado (no nulo, default 1)
• motivo (varchar(255))
• fecha_creacion (timestamp default now())
• fecha_actualizacion (timestamp)
• cancelada_por (varchar(10)) — 'paciente'|'medico'|'admin',
nullable • fecha_cancelacion (timestamp, nullable)

Restricciones:

• UNIQUE (id_horario) (un horario solo puede tener una cita activa o histórica)

auditoria_citas

• id_auditoria PK (bigserial)
• id_cita FK → citas.id_cita (no nulo)
• evento (varchar(40)) — creada, confirmada, cancelada, realizada,
no_asistio • detalle (text)
• actor_id_usuario FK → usuarios.id_usuario
• fecha_evento (timestamp default now())

tokens_recuperacion (opcional para recuperación de contraseña)

- id_token PK (bigserial)
- id_usuario FK → usuarios.id_usuario
- token (varchar(255), único)
- expira_en (timestamp)
- usado (boolean default false)

5. API REST (Backend)

Endpoints principales:

- Autenticación y usuarios

  POST /auth/register → Registrar nuevo usuario (paciente, médico o admin). POST /auth/login → Iniciar sesión y obtener token JWT.

  GET /users/{id} → Consultar perfil de usuario.

  PUT /users/{id} → Actualizar perfil.

  DELETE /users/{id} → Eliminar usuario.

- Médicos

  GET /medicos → Listar todos los médicos.

  GET /medicos/{id} → Consultar información de un médico.

  POST /medicos → Registrar un médico (solo admin).

  PUT /medicos/{id} → Actualizar datos de médico.

  DELETE /medicos/{id} → Eliminar médico (solo admin)

- Especialidades

  GET /especialidades → Listar especialidades médicas.

  POST /especialidades → Crear una nueva especialidad (solo admin).
  PUT /especialidades/{id} → Editar especialidad.

  DELETE /especialidades/{id} → Eliminar especialidad

• Citas médicas

POST /citas → Crear una cita (paciente selecciona médico, fecha y hora). GET /citas → Listar todas las citas (filtrar por paciente, médico, fecha). GET /citas/{id} → Consultar detalles de una cita. PUT /citas/{id} → Reprogramar o modificar cita. DELETE /citas/{id} → Cancelar cita

• Disponibilidad

GET /medicos/{id}/disponibilidad → Ver horarios disponibles de un médico. POST /medicos/{id}/disponibilidad → Definir disponibilidad (solo el médico o admin). PUT /medicos/{id}/disponibilidad/{idDisp} → Modificar horario. DELETE /medicos/{id}/disponibilidad/{idDisp} → Eliminar horario.

• Historial médico

GET /historial/{pacienteId} → Consultar historial clínico de un paciente. POST /historial/{pacienteId} → Agregar registro médico (solo médicos). PUT /historial/{id} → Actualizar registro. DELETE /historial/{id} → Eliminar registro (si aplica)

• Notificaciones

GET /notificaciones/{usuarioId} → Consultar notificaciones del usuario. POST /notificaciones → Crear notificación (ejemplo: recordatorio de cita). PUT /notificaciones/{id} → Marcar como

leída.

DELETE /notificaciones/{id} → Eliminar notificación

6. Documentación API

Documentar la api con swagger o postman.

7. Rúbrica de Evaluación

Criterio de Evaluación	Descripción	Peso (%)
Cumplimiento de HU y CU	Se desarrollan todas las historias de usuario y casos de uso	25%
API Backend	API implementada con rutas REST, controladores, validaciones y seguridad (JWT, bcrypt)	20%
Base de datos relacional	Modelo correcto, relaciones bien implementadas, integridad referencial.	15%
Frontend (cliente)	Consumo correcto de la API, interfaz funcional.	15%
Documentación de la API	Documentación completa de endpoints (método, ruta, parámetros, request/response).	15%
Buenas prácticas	Código limpio, modular y con control de versiones.	10%
Total		100%